8. Veri Yapıları AI Çağında Neden Hâlâ Hayati?

Array, Hash Map, Set ve Tree Arasındaki Seçim Sisteminizi Nasıl Kurtarır?
Bölüm 2 • Zorluk Derecesi: Temel / Orta Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yapay zeka her şeyi optimize edebiliyorsa, bir yazılımcının Array ile Set arasındaki bellek farkını bilmesine neden hâlâ gerek var?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Yapay zeka varsayılan olarak en kolayına gelen veri yapısını (çoğunlukla Array) seçme eğilimindedir. - 📌 Yanlış veri yapısı seçildiğinde, dünyanın en gelişmiş AI modeli bile o kodu verimli hale getiremez. - 📌 Arama için Set/Map, sıralı erişim için Array, hiyerarşik ilişkiler için Tree kullanmak mimari bir karardır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Evinizde 10.000 adet kitap olduğunu düşünün.

Bu kitapları odanın ortasına rastgele devasa bir dağ gibi yığarsanız ne olur? Belli bir kitabı bulmak için tek tek 10.000 kitabı elden geçirmeniz gerekir. İşte bu **Dizi (Array)** üzerindeki doğrusal aramadır (O(N)).

Peki bu kitapları harf sırasına göre dizilmiş kütüphane raflarına yerleştirirseniz? İstediğiniz kitaba 10 saniyede ulaşırsınız! İşte bu **Hash Map / Ağaç (Tree)** veri yapısıdır.

Yapay zeka size kitapları taşımada yardım eden hızlı bir işçidir. Ama kitapları odaya yığacak mısınız yoksa raflara mı dizeceksiniz? Bu kararı sadece siz verebilirsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Hangi Veri Yapısı Ne Zaman Seçilmeli?

Veri Yapısı	Arama Hızı	Ekleme/Silme	En İyi Senaryo	Dikkat
Array / List	O(N)	Sona ekleme O(1), Araya ekleme O(N)	Sıralı listeleme, indeksle doğrudan erişim	Büyük listelerde `includes()` veya `in` yapmak performansı öldürür
Set	O(1)	O(1)	Benzersizlik kontrolü, "Bu kullanıcı listede var mı?"	Sıralama garantisi yoktur
Hash Map / Dict	O(1)	O(1)	Anahtar-değer eşlemeleri, önbellek (cache), indeksleme	Bellek tüketimi diziye göre biraz daha fazladır
Tree / Graph	O(log N)	O(log N)	Hiyerarşik veri (kategori ağacı, organizasyon şeması)	Düğümleri yönetmek ve dengelemek uzmanlık ister

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kıdemli Veri Yapıları Uzmanı
# GÖREV: 500.000 aktif oturum ID'si (Session Token) içeren bir listede gelen
isteklerin geçerli olup olmadığını kontrol edeceğiz.

Lütfen bana bir veri yapısı önerisi ve karşılaştırması sun:
1. Python'da `list` kullanmak ile `set` kullanmak arasındaki bellek ve zaman farkı
nedir?
2. 500.000 eleman için sorgu süresi karşılaştırması (Benchmark) senaryosu açıkla.
3. Kodu en optimize veri yapısıyla yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü Veri Yapısı: Dizi (Array) üzerinde O(n) arama yapmak
export class NaiveSessionManager {
  private activeTokens: string[] = [];

  addToken(token: string) {
    this.activeTokens.push(token);
  }

  // 100.000 aktif kullanıcıda her istekte tüm diziyi gezer: Felaket!
  isValid(token: string): boolean {
    return this.activeTokens.includes(token);
  }
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Doğru Veri Yapısı: O(1) Set ile Anında Üyelik Kontrolü
export class OptimizedSessionManager {
  private activeTokens: Set<string> = new Set();

  addToken(token: string): void {
    this.activeTokens.add(token);
  }

  removeToken(token: string): boolean {
    return this.activeTokens.delete(token);
  }

  // 1.000.000 eleman olsa bile mikrosaniye düzeyinde O(1) yanıt
  isValid(token: string): boolean {
    return this.activeTokens.has(token);
  }

  get count(): number {
    return this.activeTokens.size;
  }
}
```

💡 **Kod Analizi:** Array üzerindeki `.includes()` araması O(N) iken, Set üzerindeki `.has()` kontrolü O(1) zaman karmaşıklığındadır. 100.000 öğede bu fark 50 milisaniyeden 0.001 milisaniyeye (50.000 kat) hızlanma anlamına gelir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Sadece varlık kontrolü (membership check) yaparken Array kullanmak.
  - **Çözüm:** Elemanın varlığını sorguluyorsanız her zaman Set veya Map tercih edin.

- **Tuzak:** Hash Map anahtarlarında çok büyük ve karmaşık objeler kullanıp hash çarpışması yaratmak.
  - **Çözüm:** Anahtar olarak her zaman benzersiz string veya ID kullanın.
- **Tuzak:** AI'a 'bunu hızlı yap' deyip arka plandaki veri yapısını incelememek.
  - **Çözüm:** AI'ın seçtiği veri tipinin zaman ve alan karmaşıklığını (Big-O) mutlaka sorgulayın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 8: Veri Yapıları Ustalığı - Doğru Koleksiyonu Seçmek*

- 🖥️ **Slayt 1:** Neden AI bile kötü bir veri yapısı seçimini kurtaramaz?
- 🖥️ **Slayt 2:** Array vs Set vs Map: 50.000 katlık performans farkı.
- 🖥️ **Slayt 3:** Bellek tüketimi vs Arama hızı dengesi.
- 🖥️ **Slayt 4:** Canlı Benchmark: 100.000 kayıt üzerinde List ve Set hız testi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Node.js veya Python'da bir micro-benchmark scripti yazın. 100.000 elemanlı bir listede `in` / `.includes()` aramasının kaç ms sürdüğünü, Set ile `.has()` aramasının kaç ms sürdüğünü canlı konsolda gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Mevcut kod tabanınızda bir döngü içinde `.includes()` veya `in` kontrolü yapılan en az bir yer bulun ve bunu `Set` kullanarak O(1)'e refactor edin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: 100.000 elemanlı bir koleksiyonda bir elemanın var olup olmadığını en hızlı şekilde kontrol etmek için hangi veri yapısı kullanılmalıdır? - A) Dizi (Array / List) - B) Hash Set - C) Bağlı Liste (Linked List) - D) Yığıt (Stack) > **Doğru Yanıt:** B > **Açıklama:** Hash Set, verinin hash koduna göre O(1) sabit zamanda doğrudan konumuna erişir; tüm elemanları gezmek zorunda kalmaz.

Soru 2: Array yerine Hash Map kullanmanın potansiyel tek dezavantajı nedir? - A) Asla çalışmaması - B) Hash tablolarının ve kova (bucket) yapısının bellekte biraz daha fazla yer kaplaması - C) Sadece Python'da geçerli olması - D) Kodun okunmasını imkansız kılması > **Doğru Yanıt:** B > **Açıklama:** Hash Map'ler hızlı erişim sağlamak için bellekten bir miktar ödün verir (Time-Space Tradeoff).